

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»**

УТВЕРЖДАЮ

Зав.кафедрой,

доцент, к. ф.-м. н.

_____ С. В. Миронов

ОТЧЕТ О ПРАКТИКЕ

студентки 2 курса 251 группы факультета КНиИТ

Ореховой Алины Сергеевны

вид практики: учебная (рассредоточенная)

кафедра: математической кибернетики и компьютерных наук

курс: 2

семестр: 1

продолжительность: 18 недель нед., с 01.09.23 г. по 12.01.24 г.

Руководитель практики от университета,

доцент, к. ф.-м. н.

А. С. Иванова

Руководитель практики от организации (учреждения, предприятия),

к. ф.-м. н., доцент

А. С. Иванова

Тема практики: «Разработка приложений Windows.Forms на языке C++ в среде Microsoft Visual Studio»

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Матричный калькулятор	5
1.1 Условие задания	5
1.2 Работа с векторами	5
1.3 Работа с матрицами	6
1.4 Вид формы в конструкторе	8
1.5 Таблица с описанием переименованных элементов формы	8
1.6 Примеры правильной и неправильной работы	8
1.7 Примеры исходного кода	10
2 Использование коллекций	14
2.1 Условие задания	14
2.2 Теоретическая справка	14
2.3 Вид формы в конструкторе	16
2.4 Таблица с описанием переименованных элементов формы	17
2.5 Примеры правильной и неправильной работы	17
2.6 Примеры исходного кода	20
ЗАКЛЮЧЕНИЕ	22
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	23
Приложение А Фрагменты кода программы «Матричный калькулятор» ...	24
Приложение Б Фрагменты кода программы «Использование коллекций в WindowsForms»	26
Приложение В Архив с отчетом о выполненной работе	27

ВВЕДЕНИЕ

Целью практики является освоение механизма построения оконного интерфейса приложений в среде Microsoft Visual Studio на языке C++/CLI с использованием .NET Framework и Windows Forms. В результате прохождения практики должны быть отработаны следующие навыки:

- создание нового проекта;
- добавление и настройка элементов управления;
- проверка пользовательского ввода данных для решения поставленной задачи, обработка ошибок ввода;
- разработка алгоритма решения поставленной задачи с использованием оконного интерфейса;
- тестирование приложения;
- документирование разработанного кода.

1 Матричный калькулятор

1.1 Условие задания

Создать приложение, реализующее основные операции с векторами и матрицами:

1. Ввод матрицы, вектора;
2. Создание матриц (единичная, матрица как набор векторов);
3. Умножение на число, вектор, матрицу;
4. Сложение/вычитание двух матриц;
5. Сложение/вычитание двух векторов;
6. Скалярное и векторное произведение двух векторов;
7. Транспонированная матрица;
8. Определитель, ранг матрицы.
9. Выводить сообщения об ошибках (ввод не числа, несоответствие размерностей)

1.2 Работа с векторами

Вектором называется направленный отрезок, для которого указаны его начало и конец. Свободный вектор — множество одинаково направленных отрезков.

Нахождение вектора по двум точкам

Если даны две точки плоскости $A(x_1, y_1)$ и $B(x_2, y_2)$, то вектор $\overline{AB}$ имеет следующие координаты:

$$\overline{AB}(x_2 - x_1, y_2 - y_1)$$

Если даны две точки плоскости в пространстве (3 координаты) $A(x_1, y_1, z_1)$ и $B(x_2, y_2, z_2)$, то вектор $\overline{AB}$ имеет следующие координаты:

$$\overline{AB}(x_2 - x_1, y_2 - y_1, z_2 - z_1)$$

То есть, из координат конца вектора нужно вычесть соответствующие координаты начала вектора.

Правило сложения векторов

Если даны векторы $\vec{v} = (v_1, v_2, v_3)$, $\vec{w} = (w_1, w_2, w_3)$, то их суммой является вектор $\vec{v} + \vec{w} = (v_1 + w_1, v_2 + w_2, v_3 + w_3)$. Аналогичное правило справедливо для суммы любого количества векторов.

Правило умножения вектора на число

Для того чтобы вектор $\vec{v} = (v_1, v_2, v_3)$ умножить на число λ , нужно каждую координату данного вектора умножить на число λ :

$$\lambda \vec{v} = (\lambda v_1, \lambda v_2, \lambda v_3)$$

Скалярное произведение

Скалярным произведением двух векторов $\vec{a}$ и $\vec{b}$ называется число, равное произведению длин этих векторов на косинус угла между ними:

$$\vec{a} \cdot \vec{b} = |\vec{a}| \cdot |\vec{b}| \cdot \cos \angle(\vec{a}, \vec{b})$$

Векторное произведение

Векторным произведением $[\vec{a} \times \vec{b}]$ неколлинеарных векторов $\vec{a}$ и $\vec{b}$, взятых в данном порядке, называется вектор $\vec{N}$, длина которого численно равна площади параллелограмма, построенного на данных векторах; вектор $\vec{N}$ ортогонален векторам $\vec{a}$ и $\vec{b}$, и направлен так, что базис $(\vec{a}, \vec{b}, \vec{N})$ имеет правую ориентацию.

1.3 Работа с матрицами

Алгоритм нахождения определителя матрицы

Определитель матрицы — это некоторое число, с которым можно сопоставить любую квадратную матрицу $A = (a_{ij})_{n \times n}$.

$|A|$, Δ , $\det A$ — символы, которыми обозначают определитель матрицы.

Определитель матрицы второго порядка можно вычислить по формуле:

$$|A| = \begin{vmatrix} a_1 & a_2 \\ a_3 & a_4 \end{vmatrix} = a_1 \times a_4 - a_2 \times a_3$$

Определитель матрицы третьего порядка можно вычислить по формуле:

$$|A| = \begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix} = a_{11}a_{22}a_{33} + a_{31}a_{12}a_{23} + a_{21}a_{32}a_{13} - a_{31}a_{22}a_{13} - a_{21}a_{12}a_{33} - a_{11}a_{23}a_{32}$$

Алгоритм нахождения ранга матрицы

Минор k-ого порядка матрицы — определитель квадратной матрицы по-

рядка $k \times k$, которая составлена из элементов матрицы A , находящихся в заранее выбранных k -строках и k -столбцах, при этом сохраняется положение элементов матрицы A .

Проще говоря, если в матрице A вычеркнуть $(p-k)$ строк и $(n-k)$ столбцов, а из тех элементов, которые остались, составить матрицу, сохраняя расположение элементов матрицы A , то определитель полученной матрицы и есть минор порядка k матрицы A .

Ранг матрицы — наивысший порядок матрицы, отличный от нуля.

Метод перебора миноров — метод, основанный на определении ранга матрицы по определению.

Алгоритм действий способом перебора миноров:

Необходимо найти ранг матрицы A порядка $p \times n$. При наличии хотя бы одного элемента, отличного от нуля, то ранг матрицы как минимум равен единице (т.к. есть минор 1-го порядка, который не равен нулю).

Далее следует перебор миноров 2-го порядка. Если все миноры 2-го порядка равны нулю, то ранг равен единице. При существовании хотя бы одного не равного нулю минора 2-го порядка, необходимо перейти к перебору миноров 3-го порядка, а ранг матрицы, в таком случае, будет равен минимум двум.

Аналогичным образом поступим с рангом 3-го порядка: если все миноры матрицы равняются нулю, то ранг будет равен двум. При наличии хотя бы одного ненулевого минора 3-го порядка, то ранг матрицы равен минимум трем. И так далее, по аналогии.

Так же возможно вычисление ранга матрицы методом окаймляющих миноров и методом Гаусса.

1.4 Вид формы в конструкторе

Форма имеет вид:

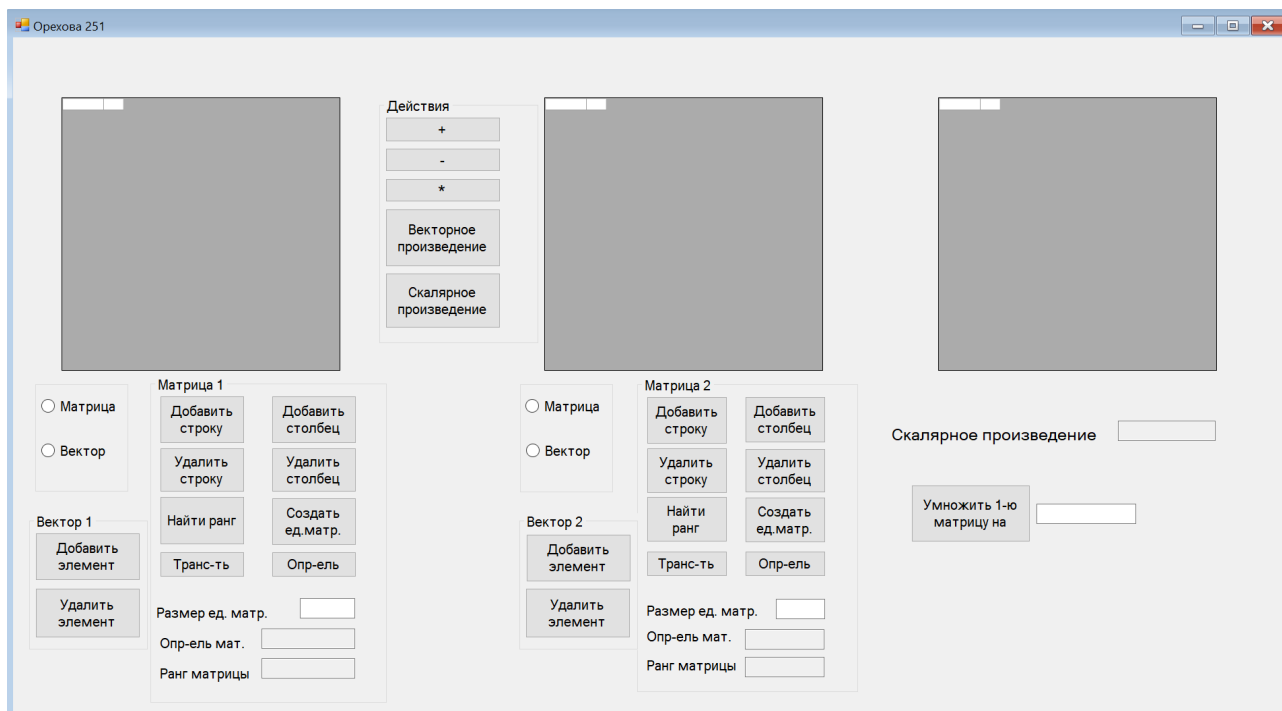


Рисунок 1.1 – Форма матричного калькулятора, открытая в конструкторе

1.5 Таблица с описанием переименованных элементов формы

Все элементы формы были переименованы и их атрибуты изменены. Проведенные изменения представлены в таблице 1.2.

1.6 Примеры правильной и неправильной работы

При запуске приложения на экране появляется окно (рис. 1.2).

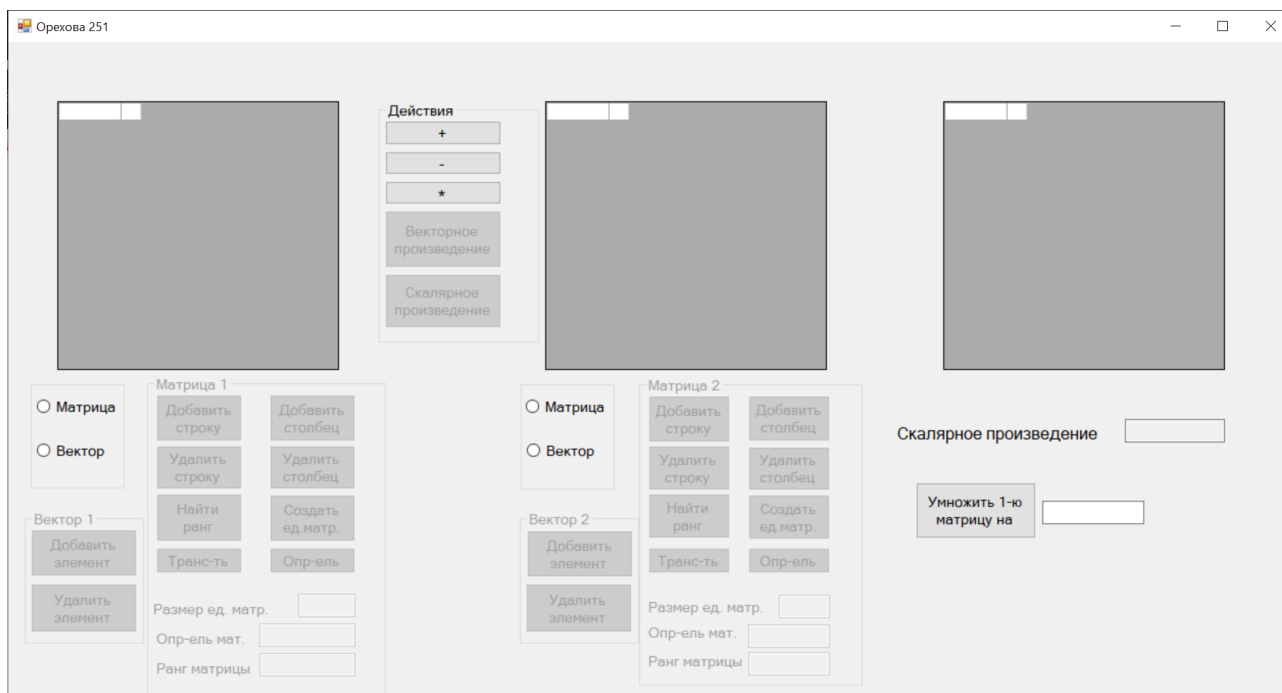


Рисунок 1.2 – Запуск программы

При вводе данных, и нажатии на кнопку "*" (рис. 1.3).

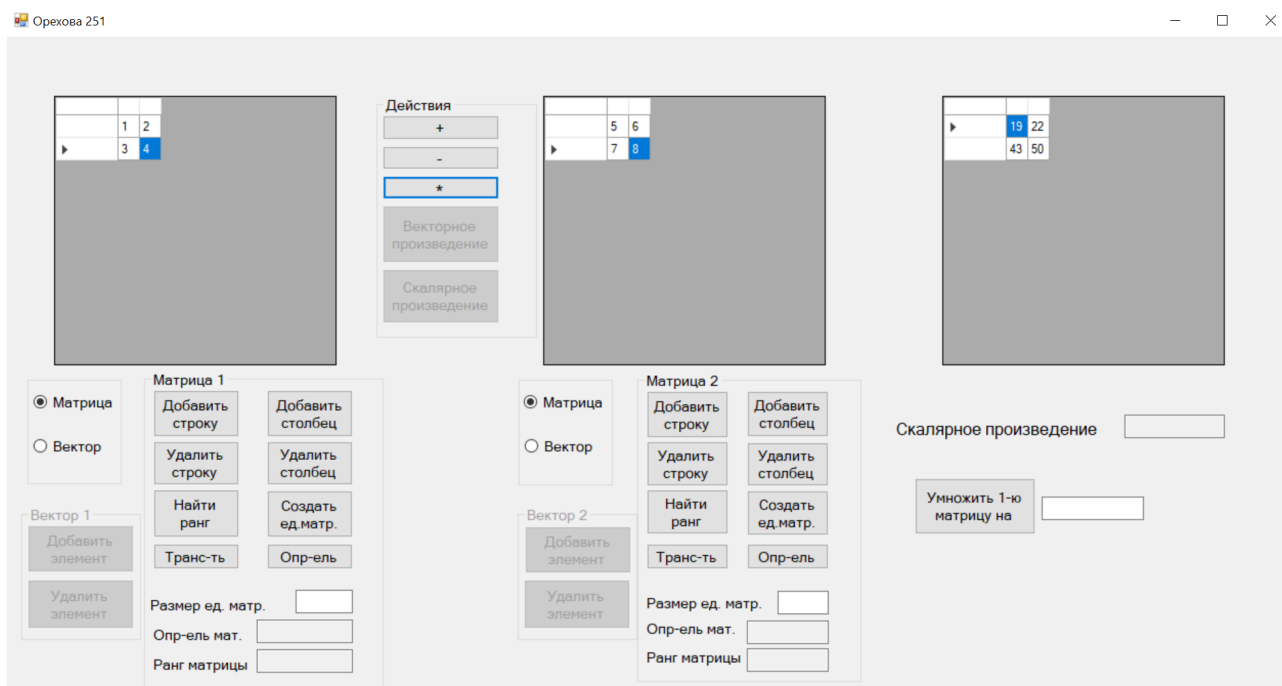


Рисунок 1.3 – Запуск программы

При вводе не числа или нецелого числа, программа выведет сообщение об этом (рис. 1.4).

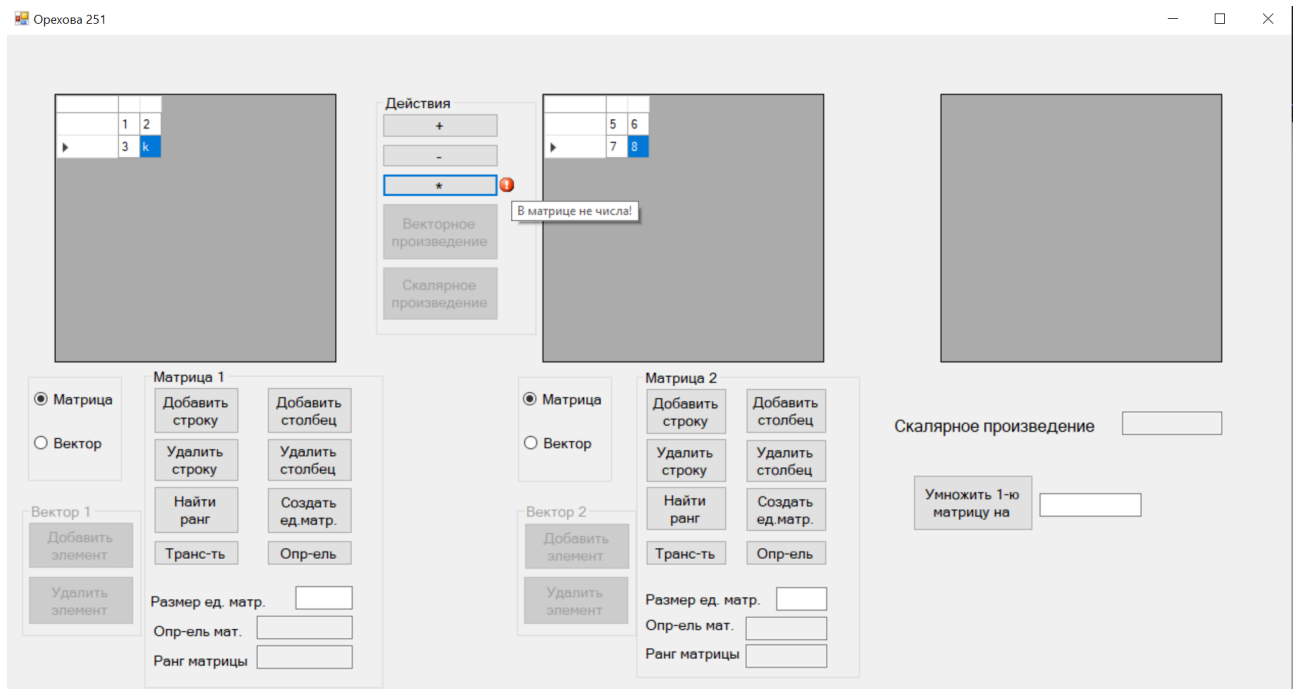


Рисунок 1.4 – Запуск программы

1.7 Примеры исходного кода

Обработчик кнопки "*":

```

1 private: System::Void bmult_Click_1(System::Object^ sender,
2   ↳ System::EventArgs^ e) {
3
4       this->errorProvider1->SetError(this->btnMultiply,
5   ↳ String::Empty);
6
7       clearMatrix(dGridRES);
8
9       if (this->dGridA->ColumnCount != this->dGridB->RowCount) {
10          this->errorProvider1->SetError(this->btnMultiply,
11   ↳ "Умножение невозможно!");
12          return;
13      }
14
15      if (!checkMatrix(dGridA) || !checkMatrix(dGridB)) {
16          this->errorProvider1->SetError(this->btnMultiply, "B
17   ↳ матрице не числа!");
18          return;
19      }

```

```

18         int n = this->dGridA->RowCount;
19         int m = this->dGridB->ColumnCount;
20
21         resize(dGridRES, n, m);
22
23         double** array = new double* [n];
24         for (int i = 0; i < n; array[i] = new double[m], ++i);
25
26         for (int i = 0; i < n; ++i)
27             for (int j = 0; j < m; ++j)
28                 array[i][j] = 0;
29
30
31
32         double** array_1 = convert(this->dGridA);
33         double** array_2 = convert(this->dGridB);
34
35
36         // умножение
37         for (int i = 0; i < this->dGridA->RowCount; ++i)
38             for (int j = 0; j < this->dGridB->ColumnCount; ++j)
39                 for (int k = 0; k < this->dGridB->RowCount;
↪ ++k)
40                     array[i][j] += array_1[i][k] *
↪ array_2[k][j];
41
42         for (int i = 0; i < n; ++i)
43             for (int j = 0; j < m; ++j)
44                 this->dGridRES->Rows[i]->Cells[j]->Value =
↪ array[i][j];
45     }

```

Другие фрагменты кода расположены в приложении А. Полный код программы приведен в приложении В.

Описание элементов формы	Список измененных атрибутов	Новое значение атрибута
Форма	Text	Сахнов 251
RadioButton "Матрица" слева	Name	rBM1
RadioButton "Матрица" справа	Name	rBM2
RadioButton "Вектор" слева	Name	rBV1
RadioButton "Вектор" справа	Name	rBV2
GroupBox Вектор 1	Name	groupBoxV1
Кнопка "Добавить элемент"	Name	btnAddElemV1
Кнопка "Удалить элемент"	Name	btnDelElemV1
GroupBox Вектор 2	Name	groupBoxV2
Кнопка "Добавить элемент"	Name	btnAddElemV2
Кнопка "Удалить элемент"	Name	btnDelElemV2
GroupBox Матрица 1	Name	groupBoxM1
Кнопка "Добавить строку"	Name	addRow1
Кнопка "Удалить строку"	Name	btnDelRow1
Кнопка "Добавить столбец"	Name	addColumn1
Кнопка "Удалить столбец"	Name	btnDelColumn1

Таблица 1.1 – Значения атрибутов элементов в приложении Матричный калькулятор

Описание элементов формы	Список измененных атрибутов	Новое значение атрибута
Кнопка "Найти ранг"	Name	btnRang1
Кнопка "Создать. ед. матр"	Name	btnMakeMX1
Кнопка "Транспонировать"	Name	btnTrans1
Кнопка "Определитель"	Name	btnDet1
Кнопка +	Name	btnPlus
Кнопка -	Name	btnMinus
Кнопка *	Name	btnMultiply
Кнопка "Векторное произведение"	Name	bvec
Кнопка "Скалярное произведение"	Name	bscal
Матрица 1	Name	dGridA
	AllowUserToAddRows	False
	AllowUserToDeleteRows	False

Таблица 1.2 – Продолжение таблицы 5.1

2 Использование коллекций

2.1 Условие задания

Разработать приложение в соответствии со своим вариантом. Номер задания выбирается преподавателем.

Приложение должно выполнять следующие действия:

1. Заголовок формы должен отражать суть задания.
2. Все элементы формы должны быть внятно подписаны (кнопки подписаны, у тестового поля должно быть написано, для чего оно нужно и т. д.)
3. В коде должны быть комментарии и отступы (код должен быть легко читаем).
4. В коде программы все элементы формы должны быть переименованы (btnName — для кнопок, lblName — для ссылок, txtName — для текстового поля и т. д.) Наименования должны быть понятными.
5. Должна быть возможность для ввода и вывода первоначальных данных.
6. Должна быть возможность для вставки и удаления одного элемента.
7. Должны использоваться коллекции.
8. Ответы на задания должны быть в разных полях.
9. Если нет данных для выполнения задания, выводить соответствующие данные.
10. Для графов использовать список смежности.

Задание 7: Создать двусвязный список, состоящий из целых чисел. Предусмотреть возможность создания списка из набора чисел, добавления одного элемента, удаления одного элемента, вывода результата на экран, удаления всех элементов с помощью кнопок. Найти сумму элементов, больших минимального нечетного элемента. Получить новый список, удалив все максимальные элементы.

2.2 Теоретическая справка

Двусвязный (двунаправленный) список — это разновидность связного списка, при которой переход по элементам возможен в обоих направлениях (как вперед, так и назад), в отличие от односвязного (однаправленного) списка.

Вот термины, необходимые для понимания концепции двусвязных (двунаправленных) списков:

1. Ссылка. В каждой ссылке связного списка могут храниться данные, назы-

ваемые элементом.

2. Следующая ссылка. В каждой ссылке связного списка содержится ссылка на следующую ссылку (Next).
3. Предыдущая ссылка. В каждой ссылке связного списка содержится ссылка на предыдущую ссылку (Prev).
4. Связный список содержит ссылку (связку) на первую ссылку (First) и на последнюю ссылку (Last).

Представление двусвязного (двунаправленного) списка

Здесь надо учитывать следующие важные моменты:

1. Двусвязный (двунаправленный) список содержит элемент ссылок — first (первой) и last (последней).
2. Каждая ссылка имеет поле или поля данных и два поля ссылки — next (следующей) и prev (предыдущей).
3. Каждая ссылка связана со следующей посредством своей ссылки next.
4. Каждая ссылка связана с предыдущей посредством своей ссылки prev.
5. Последняя ссылка содержит ссылку со значением null, обозначающую конец списка.

Базовые операции

Это основные операции, проводимые над списками:

1. Вставка, то есть добавление элемента в начало списка.
2. Удаление элемента из начала списка.
3. Вставка последним, то есть добавление элемента в конец списка.
4. Удаление последнего, то есть удаление элемента из конца списка.
5. Вставка после, то есть добавление элемента после какого-то элемента списка.
6. Удаление элемента из списка по заданному ключу.
7. Отображение вперед полного списка в прямом порядке.
8. Отображение назад полного списка в обратном порядке.

2.3 Вид формы в конструкторе

Форма имеет вид:

The screenshot shows a software constructor window titled "Орехова 251". Inside the window, there is a form with the following elements:

- A text input field at the top left.
- A button labeled "Создать список" (Create list) to the right of the first input field.
- A second text input field below the first one.
- Two buttons, "добавить" (add) and "удалить" (delete), to the right of the second input field.
- Two buttons, "Найти сумму" (Find sum) and "Новый список" (New list), arranged horizontally below the second input field.
- A block of text in the center: "Your selection: Создать двусвязный список, состоящий из целых чисел. Предусмотреть возможность создания списка из набора чисел, добавления одного элемента, удаления одного элемента, вывода результата на экран, удаления всех элементов с помощью кнопок. Найти сумму элементов, больших минимального нечетного элемента. Получить новый список, удалив все максимальные элементы."
- A label "Текущий список" (Current list) above a large, empty rectangular area at the bottom.

Рисунок 2.1 – Форма приложения, открытая в конструкторе

Описание элементов формы	Список измененных атрибутов	Новое значение атрибута
Форма	Text	Сахнов 251
TextBox Ввод списка	Name	InputTextBox
Кнопка "Создать список"	Name	createBtn
	Text	Создать список
Кнопка "add"	Name	addBtn
	Text	add
Кнопка "del"	Name	delBtn
	Text	del
Кнопка "Найти сумму"	Name	sumBtn
	Text	Найти сумму
Кнопка "Новый список"	Name	newListBtn
	Text	Новый список
TextBox Вывод	Name	output

Таблица 2.1 – Значения атрибутов элементов в приложении Коллекции

2.4 Таблица с описанием переименованных элементов формы

Все элементы формы были переименованы и их атрибуты изменены. Проведенные изменения представлены в таблице 2.1.

2.5 Примеры правильной и неправильной работы

При запуске приложения на экране появляется окно (рис. 2.2).

Создать список

добавить удалить

Найти сумму Новый список

Your selection: Создать двусвязный список, состоящий из целых чисел.
Предусмотреть возможность создания списка из набора чисел,
добавления одного элемента, удаления одного элемента, вывода
результата на экран, удаления всех элементов с помощью кнопок.
Найти сумму элементов, больших минимального нечетного элемента.
Получить новый список, удалив все максимальные элементы.

Текущий список

Рисунок 2.2 – Запуск программы

При вводе чисел через пробел и нажатии на кнопку "Создать список"(рис. 2.3).

The screenshot shows a web application interface for creating and managing a list of integers. At the top, there is a text input field containing the numbers "4 5 6 7 3 5 4 6" and a button labeled "Создать список". Below this, there is another text input field, a button labeled "добавить", and a button labeled "удалить". Further down, there are two buttons: "Найти сумму" and "Новый список". Below the buttons, there is a text block that reads: "Your selection: Создать двусвязный список, состоящий из целых чисел. Предусмотреть возможность создания списка из набора чисел, добавления одного элемента, удаления одного элемента, вывода результата на экран, удаления всех элементов с помощью кнопок. Найти сумму элементов, больших минимального нечетного элемента. Получить новый список, удалив все максимальные элементы." Below this text, there is a label "Текущий список" and a large text area displaying the current list: "4 5 6 7 3 5 4 6".

4 5 6 7 3 5 4 6

Создать список

добавить

удалить

Найти сумму

Новый список

Your selection: Создать двусвязный список, состоящий из целых чисел.
Предусмотреть возможность создания списка из набора чисел,
добавления одного элемента, удаления одного элемента, вывода
результата на экран, удаления всех элементов с помощью кнопок.
Найти сумму элементов, больших минимального нечетного элемента.
Получить новый список, удалив все максимальные элементы.

Текущий список

4 5 6 7 3 5 4 6

Рисунок 2.3 – Создание списка

При попытке добавить нецелое число, программа выведет сообщение об этом (рис. 2.4).

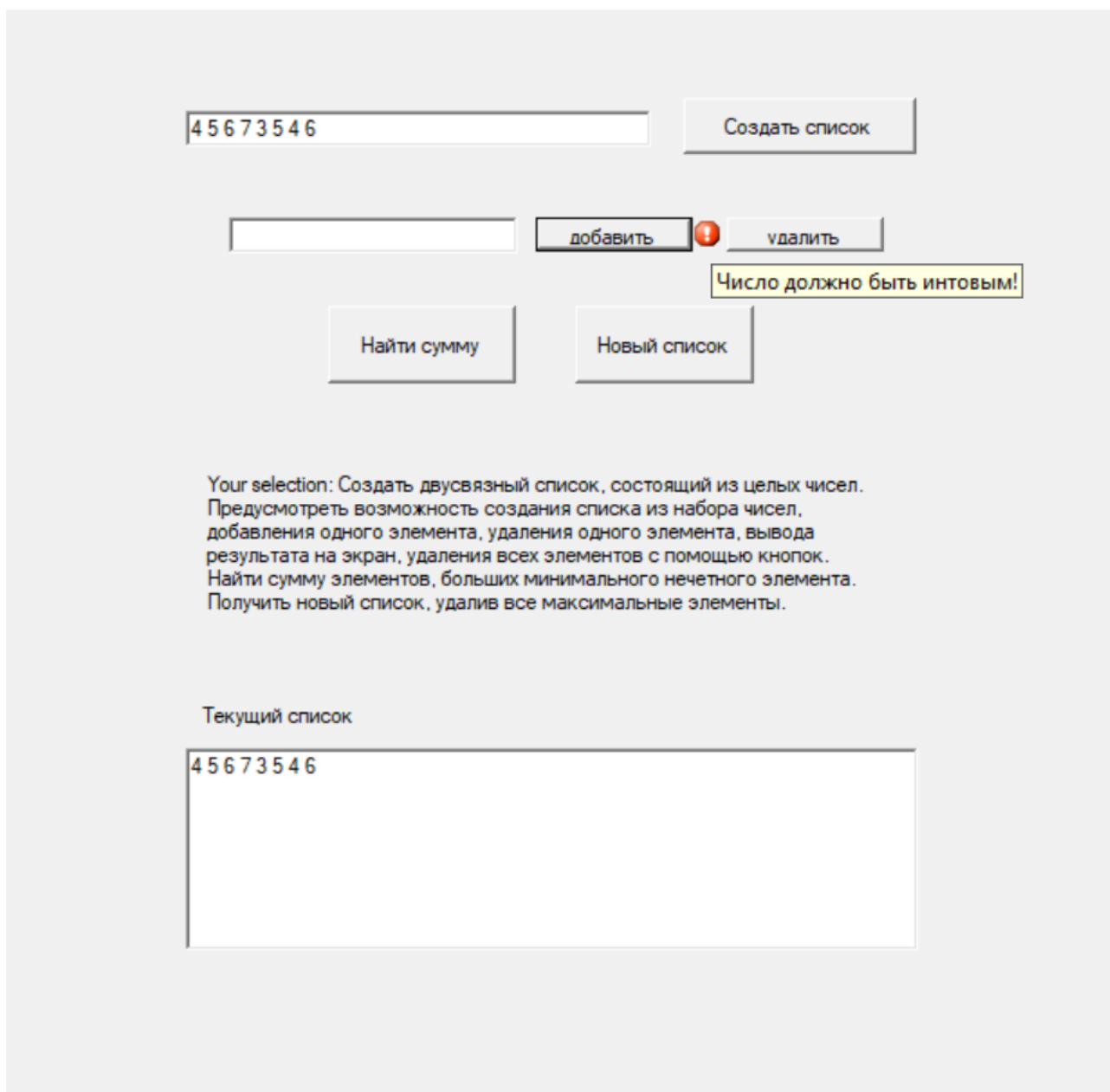


Рисунок 2.4 – Вывод сообщения об ошибке

2.6 Примеры исходного кода

Обработчик кнопки "Найти сумму":

```

1 {private: System::Void button4_Click(System::Object^ sender,
  ↳ System::EventArgs^ e) {
2         System::Collections::Generic::LinkedListNode<int>^ node =
  ↳ lst.First;
3         int min = 1000000;
4         while (node != nullptr) {
5             int temp = node->Value;
6             if (temp < min && temp % 2 != 0) {

```

```

7             min = temp;
8         }
9         node = node->Next;
10    }
11
12    node = lst.First;
13    int sum = -1;
14    while (node != nullptr) {
15        int temp = node->Value;
16        if (temp > min) {
17            sum += temp;
18        }
19        node = node->Next;
20    }
21
22
23    // обработка случая, элементов нет
24    if (sum != -1) {
25        this->textBox3->Text = System::Convert::ToString(sum);
26    }
27    else {
28        this->textBox3->Text = "Таких элементов нет";
29    }
30
31
32    }}

```

Другие фрагменты кода расположены в приложении Б. Полный код программы приведен в приложении В.

ЗАКЛЮЧЕНИЕ

В ходе практики было реализовано несколько приложений в среде Microsoft Visual Studio с целью закрепления навыков построения оконного интерфейса и программирования с использованием C++/CLI.

На практике разработаны приложения, содержащие такие элементы интерфейса, как TextBox, Label, Button, DataGridView, OpenFileDialog, SaveFileDialog, ErrorProvider.

Было освоено динамическое создание элементов формы в процессе выполнения программы, изучены виды панелей и способы размещения элементов в них.

Также были применены навыки программирования на основе обратных вызовов при обработке событий формы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 *Strauss, D.* Getting to know visual studio 2022 / D. Strauss // Getting Started with Visual Studio 2022. — Springer, 2023. — Pp. 1–64.
- 2 *Powers, L.* Microsoft visual studio 2008 Unleashed / L. Powers, M. Snell. — Pearson Education, 2008.
- 3 *Gladstone, A.* Building a c++/cli wrapper / A. Gladstone // C++ Software Interoperability for Windows Programmers. — Springer, 2022. — Pp. 41–66.
- 4 *Johnson, B.* Professional visual studio 2012 / B. Johnson. — John Wiley & Sons, 2012.
- 5 *Лаврентьев, А. О.* Использование технологии windows forms для разработки информационных систем (на примере электронного журнала) / А. О. Лаврентьев, В. К. Белаш // *Вестник Калужского университета*. — 2020. — no. 4. — Pp. 82–85.
- 6 *Kaiser, R.* C++/cli, net-bibliotheken und c++ interoperabilität / R. Kaiser // C++ mit Visual Studio 2022 und Windows Forms-Anwendungen. — Springer, 2022. — Pp. 937–974.
- 7 *Комракова, Е. В.* Приложение windows forms для формирования рейтинга и отзывов к японской мультипликации // *Современная наука: актуальные вопросы, достижения и инновации*. — 2022. — Pp. 19–21.
- 8 *Fraser, S. R. G.* Advanced windows forms applications / S. R. G. Fraser // *Pro Visual C++/CLI and the .NET 2.0 Platform*. — 2006. — Pp. 377–444.
- 9 *Griffiths, I.* Mastering Visual Studio. NET / I. Griffiths, J. Flanders, C. Sells. — "O'Reilly Media, Inc. 2003.
- 10 *Chang, G. W.* Analysis of paint messages with multiple c++/cli controls // 2021 International Conference on Electronic Communications, Internet of Things and Big Data (ICEIB) / IEEE. — 2021. — Pp. 138–141.

ПРИЛОЖЕНИЕ А

Фрагменты кода программы «Матричный калькулятор»

Код создания единичной матрицы:

```
1  //создаем единичную матрицу
2      private: System::Void btnMakeMX1_Click(System::Object^ sender,
↪   System::EventArgs^ e) {
3          this->errorProvider1->SetError(this->txtSize1, String::Empty);
4          ClearInput1();
5          int size;
6          bool resSize = Int32::TryParse(this->txtSize1->Text, size);
7
8          if (!resSize) {
9              this->errorProvider1->SetError(this->txtSize1,
↪   "Введите число!");
10             return;
11         }
12
13         for (int i = 0; i < size-1; ++i) {
14             addColumn(this->dGridA);
15             addRow(this->dGridA);
16         }
17         addRow(this->dGridA);
18
19         for (int i = 0; i < size; ++i)
20             for (int j = 0; j < size; ++j)
21                 this->dGridA->Rows[i]->Cells[j]->Value =
↪   System::Convert::ToInt32(i == j);
22     }
```

Сложение:

```
1      // функция сложения
2      private: void add(int flag) {
3          if (flag > 0)
4              this->errorProvider2->SetError(this->btnPlus,
↪   String::Empty);
5          else
6              this->errorProvider2->SetError(this->btnMinus,
↪   String::Empty);
7          clearMatrix(dGridRES);
8      }
```



```

9             if (!checkMatrix(dGridA) || !checkMatrix(dGridB)) {
10                 if (flag > 0)
11                     this->errorProvider2->SetError(this->btnPlus,
↪     "В матрице должны быть числа");
12                 else
13                     this->errorProvider2->SetError(this->btnMinus
↪     "В матрице должны быть числа");
14                 return;
15             }
16
17             if (dGridA->RowCount != dGridB->RowCount ||
↪     dGridA->ColumnCount != dGridB->ColumnCount) {
18                 if (flag > 0)
19                     this->errorProvider2->SetError(this->btnPlus,
↪     "Сложение невозможно!");
20                 else
21                     this->errorProvider2->SetError(this->btnMinus
↪     "Сложение невозможно!");
22
23                 return;
24             }
25
26             double** array_1 = convert(dGridA);
27             double** array_2 = convert(dGridB);
28
29             int n = this->dGridA->RowCount;
30             int m = this->dGridB->ColumnCount;
31
32             resize(dGridRES, n, m);
33
34             for (int i = 0; i < n; ++i)
35                 for (int j = 0; j < m; ++j) {
36                     array_1[i][j] += array_2[i][j] * flag;
37                     this->dGridRES->Rows[i]->Cells[j]->Value
↪     = array_1[i][j];
38                 }
39             }

```

ПРИЛОЖЕНИЕ Б

Фрагменты кода программы «Использование коллекций в WindowsForms»

Создание списка:

```
1      //кнопка создания списка
2      private: System::Void button1_Click(System::Object^ sender,
↪      System::EventArgs^ e) {
3          System::String^ str =
↪      System::Convert::ToString(this->textBox1->Text);
4          str += " ";
5          int x, at, pos = 0;
6          at = str->IndexOf(" ");
7          while (at != -1) {
8              System::String^ str1 = str->Substring(pos, at - pos);
9              pos = at + 1;
10             bool res = Int32::TryParse(str1, x);
11             lst.AddLast(x);
12             at = str->IndexOf(" ", pos);
13         }
14         this->textBox3->Text = str;
```

Доп функция, для перевода списка в строку:

```
1      private: System::String^ lstToString() {
2          System::Collections::Generic::LinkedListNode <int>^ node =
↪      lst.First;
3          System::String^ str = "";
4          while (node != nullptr) {
5              str += node->Value.ToString();
6              str += " ";
7              node = node->Next;
8          }
9          return str;
10
11      }
```

ПРИЛОЖЕНИЕ В

Архив с отчетом о выполненной работе

В приложенном ZIP-архиве, для ознакомления, содержатся следующие файлы:

Папка report — LATEX-версия отчета о практике;

Папка task-1 — задание №1;

Папка task-2 — задание №2, вариант 2;

Папка task-3 — задание №3, вариант 3;

Папка task-4 — задание №4, вариант 12;

Папка task-5 — задание №5, вариант 9;

Папка task-6 — задание №6 (матричный калькулятор);

Папка task-7 — задание №7, вариант 9;

Папка task-8 — задание №8, вариант 9;

Папка task-9 — задание №9;

Report.pdf — отчет о практике.